
PerceptionLab - QA & BDD Specification

Quality Assurance, Behavior-Driven Development and database consistency plan

Jérémie Nunez

2026-06-15

Executive summary

1. Objectif QA
2. Stratégie QA globale
3. Niveaux de criticité
4. Conventions BDD
5. Données de test
6. Quality gates
 - Gate PR
 - Gate integration
 - Gate MVP
7. Matrice de traçabilité
8. Feature - Healthcheck
9. Feature - Dataset Management
10. Feature - Sample Ingestion
11. Feature - Annotation Management
12. Feature - Dataset Versioning
13. Feature - Training Jobs
14. Feature - Metrics Tracking
15. Feature - Model Registry
16. Feature - Inference API
17. Feature - Model Export

18. Feature - Visual Overlay
19. Feature - Artifacts Storage
20. Feature - Database Integrity
21. Feature - Observability
22. Feature - Performance Smoke
23. Scénario end-to-end principal
24. Test BDD anti-régression critique
25. Contrats d'erreur standardisés
26. Contrôles spécifiques ML
 - QA ML MVP
 - QA ML V2
27. QA base de données
28. QA storage
29. QA worker
30. QA sécurité MVP
31. Definition of Done QA
32. Plan d'exécution QA par sprint
 - Sprint 1 - API foundation QA
 - Sprint 2 - Data ingestion QA
 - Sprint 3 - Versioning + Training QA
 - Sprint 4 - Registry + Inference QA
 - Sprint 5 - Export + Polish QA
33. Résultat attendu pour le repo
34. Synthèse

Executive summary

PerceptionLab is a Rust + PyTorch ML infrastructure project. The QA and BDD strategy must prove that the platform works end to end: raw visual data can be ingested, annotated, versioned, used by asynchronous training jobs, converted into registered model artifacts, served through inference, exported, and visualized through overlays.

The testing strategy is intentionally broader than endpoint testing. It validates API behavior, business rules, database integrity, storage consistency, training lifecycle, worker execution, model registry correctness, inference contracts, export artifacts, observability and minimal performance expectations.

The main acceptance sentence for the product is:

I can ingest visual data, annotate it, version it, train a model, register it, run inference, and generate a visual result - all through a clean Rust + PyTorch platform.

1. Objectif QA

La QA de PerceptionLab doit prouver que la plateforme fonctionne de bout en bout:

Créer dataset
-> ingérer images
-> ajouter annotations
-> figer dataset version
-> lancer training job
-> suivre métriques
-> créer modèle
-> exécuter inférence
-> exporter modèle
-> générer overlay visuel

L'objectif n'est pas seulement de vérifier des endpoints. L'objectif est de garantir que le produit se comporte comme une vraie plateforme de ML infrastructure.

La QA doit donc couvrir:

API behavior
Business rules
Database integrity
Storage consistency
Training job lifecycle
Worker behavior
Model registry correctness
Inference response contract
Export artifacts
Error handling
Observability
Minimal performance

2. Stratégie QA globale

La pyramide de test recommandée:

```
Unit tests
↓
Integration tests
↓
API contract tests
↓
BDD acceptance tests
↓
End-to-end demo tests
↓
Performance smoke tests
```

Les tests unitaires valident les fonctions isolées: validation bbox, validation payload, conversion annotation, calcul checksum, mapping statuts.

Les tests d'intégration valident les interactions: API + PostgreSQL, API + storage, worker + queue, worker + DB, worker + artifacts.

Les tests BDD valident les comportements utilisateur avec des scénarios lisibles en Gherkin.

Les tests end-to-end valident le scénario complet du produit.

Les tests performance smoke vérifient que la plateforme ne s'écroule pas sur les opérations essentielles: upload concurrent, création de job rapide, inférence acceptable.

3. Niveaux de criticité

Les scénarios sont classés en quatre priorités:

```
P0 - Obligatoire MVP
P1 - Important pour crédibilité GitHub
P2 - Avancé / V2
P3 - Bonus
```

Pour considérer le MVP comme validé:

```
100% des scénarios P0 doivent être verts.
Au moins 80% des scénarios P1 doivent être verts.
Aucun bug critique ne doit exister sur ingestion, versioning, training jobs ou model registry.
```

4. Conventions BDD

Les feature files seront écrites en anglais pour rester propres dans un repo GitHub technique.

Structure recommandée:

```
qa/
  features/
    health.feature
    dataset_management.feature
    sample_ingestion.feature
    annotation_management.feature
    dataset_versioning.feature
    training_jobs.feature
    metrics_tracking.feature
    model_registry.feature
    inference_api.feature
    model_export.feature
```

```
visual_overlay.feature
artifacts_storage.feature
database_integrity.feature
observability.feature
performance_smoke.feature

fixtures/
  images/
    cup.jpg
    book.jpg
    invalid.txt
    oversized.jpg
  annotations/
    desk_objects_yolo.zip
  payloads/
    create_dataset.json
    create_training_job.json

reports/
  bdd-report.html
  junit.xml
```

Tags recommandés:

```
@p0
@p1
@p2
@api
@database
@storage
@worker
@ml
@inference
@export
@nonfunctional
```

5. Données de test

Dataset principal de test:

```
Dataset name: desk-objects-v1
Task type: object_detection
Classes:
- cup
- book
- phone
- keyboard
- mouse
```

Images de test:

```
cup.jpg - image valide contenant une tasse
book.jpg - image valide contenant un livre
phone.jpg - image valide contenant un téléphone
invalid.txt - fichier invalide
oversized.jpg - image dépassant la limite configurée
corrupted.jpg - image avec extension valide mais contenu corrompu
```

Annotations de test:

```
{
  "class_name": "cup",
  "bbox": {
```

```
"x": 0.35,  
"y": 0.42,  
"width": 0.22,  
"height": 0.28  
}  
}
```

Hyperparamètres de training MVP:

```
{  
  "epochs": 2,  
  "batch_size": 2,  
  "image_size": 320,  
  "learning_rate": 0.001  
}
```

Pour la CI, le training doit avoir deux modes:

```
fake_training - rapide, déterministe, utilisé dans les tests BDD CI  
tiny_training - vrai PyTorch sur petit dataset, utilisé en local ou nightly
```

6. Quality gates

Gate PR

Avant merge d'une PR:

```
cargo fmt  
cargo clippy  
cargo test  
python lint  
python unit tests  
database migration check  
API contract smoke
```

Gate integration

Avant validation d'un module:

```
docker compose up  
migrations PostgreSQL OK  
storage disponible  
queue disponible  
API health OK  
worker health OK  
tests BDD P0 du module OK
```

Gate MVP

Avant annoncer le MVP:

```
Tous les scénarios P0 passent.  
Le scénario end-to-end complet passe.  
Le README permet de reproduire la démo.  
Un overlay image est généré.
```

Un modèle est référencé dans le registry.
Un export ONNX est disponible ou mocké proprement si P1 non terminé.

7. Matrice de traçabilité

Product module	Feature file	Priorité
Healthcheck	health.feature	P0
Dataset Management	dataset_management.feature	P0
Sample Ingestion	sample_ingestion.feature	P0
Annotation Management	annotation_management.feature	P0
Dataset Versioning	dataset_versioning.feature	P0
Training Jobs	training_jobs.feature	P0
Metrics Tracking	metrics_tracking.feature	P0
Model Registry	model_registry.feature	P0
Inference API	inference_api.feature	P0
Model Export	model_export.feature	P1
Visual Overlay	visual_overlay.feature	P1
Artifacts Storage	artifacts_storage.feature	P0
Database Integrity	database_integrity.feature	P0
Observability	observability.feature	P1
Performance Smoke	performance_smoke.feature	P1

8. Feature - Healthcheck

```
@p0 @api
Feature: Platform healthcheck
  The platform must expose health endpoints so that users and automation can verify service readiness.

  Background:
    Given the PerceptionLab API is running

  Scenario: API healthcheck returns healthy status
    When I call GET "/health"
    Then the response status should be 200
    And the response body should contain "status" equal to "ok"

  Scenario: API healthcheck exposes dependency status
    When I call GET "/health"
    Then the response body should contain a "database" status
    And the response body should contain a "storage" status
    And the response body should contain a "queue" status

  Scenario: API returns degraded status when a dependency is unavailable
    Given the storage service is unavailable
    When I call GET "/health"
```

Then the response status should be 503
And the response body should contain "status" equal to "degraded"
And the response body should contain an explicit storage error

9. Feature - Dataset Management

@p0 @api @database

Feature: Dataset management

Users must be able to create and inspect datasets for computer vision training.

Background:

Given the PerceptionLab API is running
And the database is clean

Scenario: Create a valid object detection dataset

When I create a dataset with name "desk-objects-v1"
And task type "object_detection"
And classes:

	cup	
	book	
	phone	
	keyboard	
	mouse	

Then the response status should be 201
And the response should contain a dataset id
And the dataset status should be "draft"
And the dataset should be persisted in the database

Scenario: Reject dataset creation with an empty name

When I create a dataset with an empty name
Then the response status should be 422
And the response should contain a validation error for "name"

Scenario: Reject dataset creation with unsupported task type

When I create a dataset with task type "audio_classification"
Then the response status should be 422
And the response should contain a validation error for "task_type"

Scenario: Reject duplicate dataset name

Given a dataset named "desk-objects-v1" already exists
When I create another dataset named "desk-objects-v1"
Then the response status should be 409
And the response should explain that the dataset name already exists

Scenario: List existing datasets

Given a dataset named "desk-objects-v1" exists
And a dataset named "warehouse-objects-v1" exists
When I call GET "/datasets"
Then the response status should be 200
And the response should contain 2 datasets
And the response should contain "desk-objects-v1"
And the response should contain "warehouse-objects-v1"

Scenario: Retrieve dataset details

Given a dataset named "desk-objects-v1" exists
When I retrieve this dataset by id
Then the response status should be 200
And the response should contain the dataset name
And the response should contain the task type
And the response should contain the class list

Scenario: Retrieve dataset statistics

Given a dataset named "desk-objects-v1" exists
And the dataset contains 2 samples
And the dataset contains 3 annotations
When I call GET "/datasets/{dataset_id}/stats"
Then the response status should be 200
And the response should contain "sample_count" equal to 2
And the response should contain "annotation_count" equal to 3

10. Feature - Sample Ingestion

@p0 @api @storage @database

Feature: Sample ingestion

Users must be able to upload valid images into datasets.

Background:

Given the PerceptionLab API is running

And the storage service is running

And the database is clean

And a dataset named "desk-objects-v1" exists with classes:

	cup	
	book	
	phone	

Scenario: Upload a valid image sample

When I upload the image "cup.jpg" to the dataset

Then the response status should be 201

And the response should contain a sample id

And the sample should have a storage URI

And the sample metadata should contain width

And the sample metadata should contain height

And the sample metadata should contain checksum

And the image file should exist in storage

Scenario: Reject upload to an unknown dataset

When I upload the image "cup.jpg" to an unknown dataset id

Then the response status should be 404

And the response should explain that the dataset does not exist

Scenario: Reject unsupported file type

When I upload the file "invalid.txt" to the dataset

Then the response status should be 415

And no sample metadata should be created

And no file should be stored

Scenario: Reject corrupted image

When I upload the file "corrupted.jpg" to the dataset

Then the response status should be 422

And the response should explain that the image cannot be decoded

And no sample metadata should be created

Scenario: Reject oversized image

Given the maximum upload size is 10MB

When I upload the image "oversized.jpg" to the dataset

Then the response status should be 413

And the response should explain that the file is too large

Scenario: Detect duplicate image in the same dataset

Given I uploaded the image "cup.jpg" to the dataset

When I upload the same image "cup.jpg" again to the same dataset

Then the response status should be 409

And the response should explain that the sample already exists

And only one sample should exist for this checksum in the dataset

Scenario: Allow same image checksum in another dataset

Given I uploaded the image "cup.jpg" to dataset "desk-objects-v1"

And a dataset named "desk-objects-v2" exists

When I upload the image "cup.jpg" to dataset "desk-objects-v2"

Then the response status should be 201

And each dataset should contain one sample

11. Feature - Annotation Management

@p0 @api @database

Feature: Annotation management

Users must be able to add object detection annotations to uploaded samples.

```

Background:
  Given the PerceptionLab API is running
  And a dataset named "desk-objects-v1" exists with classes:
    | cup |
    | book |
    | phone |
  And the image "cup.jpg" has been uploaded as a sample

Scenario: Add a valid bounding box annotation
  When I add an annotation to the sample with class "cup"
  And bounding box:
    | x | y | width | height |
    | 0.35 | 0.42 | 0.22 | 0.28 |
  Then the response status should be 201
  And the response should contain an annotation id
  And the annotation should be persisted in the database

Scenario: Reject annotation with unknown class
  When I add an annotation to the sample with class "bottle"
  And bounding box:
    | x | y | width | height |
    | 0.20 | 0.20 | 0.30 | 0.30 |
  Then the response status should be 422
  And the response should contain a validation error for "class_name"
  And no annotation should be persisted

Scenario: Reject annotation with negative coordinates
  When I add an annotation to the sample with class "cup"
  And bounding box:
    | x | y | width | height |
    | -0.10 | 0.20 | 0.30 | 0.30 |
  Then the response status should be 422
  And the response should contain a validation error for "bbox"

Scenario: Reject annotation outside image bounds
  When I add an annotation to the sample with class "cup"
  And bounding box:
    | x | y | width | height |
    | 0.90 | 0.90 | 0.30 | 0.30 |
  Then the response status should be 422
  And the response should explain that the bounding box exceeds image bounds

Scenario: List annotations for a sample
  Given the sample has an annotation for class "cup"
  When I call GET "/samples/{sample_id}/annotations"
  Then the response status should be 200
  And the response should contain 1 annotation
  And the annotation class should be "cup"

Scenario: Delete an annotation
  Given the sample has an annotation for class "cup"
  When I delete this annotation
  Then the response status should be 204
  And the annotation should no longer exist in the database

```

12. Feature - Dataset Versioning

```

@p0 @api @database
Feature: Dataset versioning
  Users must be able to freeze datasets into immutable versions used by training jobs.

Background:
  Given the PerceptionLab API is running
  And a dataset named "desk-objects-v1" exists with classes:
    | cup |
    | book |
  And the dataset contains annotated samples

Scenario: Create a dataset version
  When I create a dataset version named "v1"
  Then the response status should be 201

```

And the response should contain a dataset version id
And the version should contain the current sample count
And the version should contain the current annotation count
And the version should contain a snapshot of the class list

Scenario: Dataset version is immutable after dataset changes
Given a dataset version named "v1" exists
When I upload a new image to the dataset
And I add a new annotation to the new image
Then dataset version "v1" should keep its original sample count
And dataset version "v1" should keep its original annotation count

Scenario: List dataset versions
Given dataset version "v1" exists
And dataset version "v2" exists
When I call GET "/datasets/{dataset_id}/versions"
Then the response status should be 200
And the response should contain "v1"
And the response should contain "v2"

Scenario: Reject dataset version creation with duplicate version name
Given dataset version "v1" exists
When I create another dataset version named "v1"
Then the response status should be 409
And the response should explain that the version already exists

Scenario: Training must use a dataset version, not a mutable dataset
When I create a training job using a dataset id instead of a dataset version id
Then the response status should be 422
And the response should explain that a dataset version is required

13. Feature - Training Jobs

@p0 @api @worker @ml @database

Feature: Training job orchestration

Users must be able to launch asynchronous PyTorch training jobs from dataset versions.

Background:

Given the PerceptionLab API is running
And the worker service is running
And a trainable dataset version "desk-objects-v1:v1" exists

Scenario: Create a training job from a valid dataset version

When I create a training job with model family "yolo"
And base model "yololln"
And hyperparameters:
| epochs | batch_size | image_size | learning_rate |
| 2 | 2 | 320 | 0.001 |
Then the response status should be 201
And the response should contain a training job id
And the job status should be "queued"

Scenario: Worker picks up queued training job

Given a training job exists with status "queued"
When the worker polls for available jobs
Then the job status should become "running"
And the job should have a started_at timestamp

Scenario: Successful training job creates a model

Given a training job is running in fake training mode
When the worker completes the training job successfully
Then the job status should become "succeeded"
And the job should have a finished_at timestamp
And a model should be registered for this job
And the model status should be "candidate"

Scenario: Failed training job stores an explicit error

Given a training job is running
And the worker encounters a training error
When the worker marks the job as failed
Then the job status should become "failed"

And the job should contain an error message
And no model should be registered for this job

Scenario: Reject training job with unknown dataset version
When I create a training job with an unknown dataset version id
Then the response status should be 404
And the response should explain that the dataset version does not exist

Scenario: Reject training job with invalid hyperparameters
When I create a training job with epochs equal to 0
Then the response status should be 422
And the response should contain a validation error for "epochs"

Scenario: Reject training job from an untrainable dataset version
Given a dataset version exists with no annotations
When I create a training job from this dataset version
Then the response status should be 422
And the response should explain that the dataset version is not trainable

14. Feature - Metrics Tracking

@p0 @api @worker @database @ml

Feature: Metrics tracking

Training jobs must expose metrics so users can evaluate model quality.

Background:

Given the PerceptionLab API is running
And a training job exists

Scenario: Worker writes metrics for each epoch
Given the training job is running
When the worker completes epoch 1 with metrics:
| train_loss | val_loss | precision | recall | mAP50 |
| 0.82 | 0.91 | 0.64 | 0.58 | 0.61 |
Then the metrics should be persisted for epoch 1
And the metrics should be associated with the training job

Scenario: Retrieve metrics for a training job
Given the training job has metrics for 2 epochs
When I call GET "/training-jobs/{job_id}/metrics"
Then the response status should be 200
And the response should contain 2 metric records
And the metrics should be ordered by epoch ascending

Scenario: Retrieve empty metrics for a queued job
Given the training job status is "queued"
When I call GET "/training-jobs/{job_id}/metrics"
Then the response status should be 200
And the response should contain an empty metrics list

Scenario: Training job summary exposes best epoch
Given the training job has metrics for multiple epochs
When I retrieve the training job details
Then the response should contain a metrics summary
And the metrics summary should contain the best mAP50
And the metrics summary should contain the best epoch

15. Feature - Model Registry

@p0 @api @database @storage

Feature: Model registry

Successful training jobs must produce registered models with traceable artifacts.

Background:

Given the PerceptionLab API is running
And a successful training job exists

And a model artifact exists in storage

Scenario: Register model after successful training
When the worker finalizes the successful training job
Then a model should be created in the registry
And the model should reference the training job id
And the model should reference the dataset version id
And the model should contain an artifact URI
And the model status should be "candidate"

Scenario: List registered models
Given the registry contains 2 models
When I call GET "/models"
Then the response status should be 200
And the response should contain 2 models

Scenario: Retrieve model details
Given a registered model exists
When I call GET "/models/{model_id}"
Then the response status should be 200
And the response should contain the model family
And the response should contain the training job id
And the response should contain the dataset version id
And the response should contain the metrics summary

Scenario: Failed training job does not create model
Given a training job has status "failed"
When I retrieve models for this training job
Then no model should exist for this training job

@p2

Scenario: Promote a candidate model
Given a model exists with status "candidate"
When I promote this model
Then the model status should become "promoted"
And the previous promoted model for the same dataset should no longer be promoted

16. Feature - Inference API

@p0 @api @inference @ml

Feature: Inference API

Users must be able to run inference on an image using a registered model.

Background:

Given the PerceptionLab API is running
And a registered model exists with status "candidate"
And the model artifact is available

Scenario: Run inference with a valid model and image
When I send image "cup.jpg" to POST "/models/{model_id}/infer"
Then the response status should be 200
And the response should contain "model_id"
And the response should contain "latency_ms"
And the response should contain a detections list

Scenario: Inference detection contains required fields
When I run inference on image "cup.jpg"
Then each detection should contain "class_id"
And each detection should contain "class_name"
And each detection should contain "confidence"
And each detection should contain "bbox"

Scenario: Confidence threshold filters detections
Given the inference confidence threshold is 0.90
When I run inference on image "cup.jpg"
Then every returned detection should have confidence greater than or equal to 0.90

Scenario: Reject inference with unknown model id
When I send image "cup.jpg" to an unknown model id
Then the response status should be 404
And the response should explain that the model does not exist

```
Scenario: Reject inference when model artifact is missing
  Given the model artifact is missing from storage
  When I run inference on image "cup.jpg"
  Then the response status should be 500
  And the response should contain a safe error message
  And the response should not expose internal stack traces

Scenario: Reject inference with invalid image file
  When I send file "invalid.txt" to POST "/models/{model_id}/infer"
  Then the response status should be 415
  And the response should explain that the file type is unsupported
```

17. Feature - Model Export

```
@p1 @api @export @storage @ml
Feature: Model export
  Users must be able to export registered models to deployment formats.

  Background:
    Given the PerceptionLab API is running
    And a registered model exists
    And the model artifact is available

  Scenario: Request ONNX export
    When I request an export for the model with format "onnx"
    Then the response status should be 202
    And the response should contain an export id
    And the export status should be "queued"

  Scenario: Successful ONNX export creates artifact
    Given an ONNX export is queued
    When the worker completes the export successfully
    Then the export status should become "succeeded"
    And the export should contain an artifact URI
    And the exported artifact should exist in storage

  Scenario: Retrieve exports for a model
    Given the model has an ONNX export
    When I call GET "/models/{model_id}/exports"
    Then the response status should be 200
    And the response should contain the ONNX export

  Scenario: Reject unsupported export format
    When I request an export for the model with format "tflite"
    Then the response status should be 422
    And the response should explain that the export format is unsupported

  Scenario: Failed export stores error message
    Given an export is running
    When the export process fails
    Then the export status should become "failed"
    And the export should contain an error message
```

18. Feature - Visual Overlay

```
@p1 @api @inference
Feature: Visual overlay generation
  Users must be able to generate annotated images from inference results.

  Background:
    Given the PerceptionLab API is running
    And an inference run exists with detections

  Scenario: Generate overlay from inference run
    When I request an overlay for the inference run
```

Then the response status should be 201
And the response should contain an overlay artifact URI
And the overlay image should exist in storage

Scenario: Overlay contains class name and confidence
Given the inference run contains a detection for class "cup" with confidence 0.89
When I generate an overlay
Then the overlay label should contain "cup"
And the overlay label should contain "89%"

Scenario: Overlay displays distance when depth metadata exists
Given the inference run contains a detection for class "cup"
And the detection contains distance_m equal to 0.4
When I generate an overlay
Then the overlay label should contain "cup"
And the overlay label should contain "0.4 m"

Scenario: Reject overlay generation for unknown inference run
When I request an overlay for an unknown inference run id
Then the response status should be 404
And the response should explain that the inference run does not exist

19. Feature - Artifacts Storage

@p0 @storage @database

Feature: Artifacts storage consistency

Files and database metadata must stay consistent across uploads, models, exports and overlays.

Background:

Given the PerceptionLab API is running
And the storage service is running
And the database is clean

Scenario: Sample metadata is created only after successful file storage

Given a dataset exists
And the storage service fails during upload
When I upload image "cup.jpg"
Then the response status should be 500
And no sample metadata should be created in the database

Scenario: Model artifact is stored after successful training

Given a training job completes successfully
When the worker saves the model artifact
Then the artifact should exist in storage
And the model registry should reference the artifact URI

Scenario: Export artifact is linked to the source model

Given a model exists
When an ONNX export succeeds
Then the export artifact should exist in storage
And the export should reference the source model id

Scenario: Artifact download refuses unknown artifact

When I request download for an unknown artifact id
Then the response status should be 404
And the response should explain that the artifact does not exist

20. Feature - Database Integrity

@p0 @database

Feature: Database integrity

The database must enforce referential integrity and protect core ML pipeline consistency.

Background:

Given the database schema is migrated
And the database is clean

Scenario: Sample cannot exist without dataset
 When the system attempts to create a sample with an unknown dataset id
 Then the database should reject the operation
 And no sample row should be persisted

Scenario: Annotation cannot exist without sample
 When the system attempts to create an annotation with an unknown sample id
 Then the database should reject the operation
 And no annotation row should be persisted

Scenario: Training job cannot exist without dataset version
 When the system attempts to create a training job with an unknown dataset version id
 Then the database should reject the operation
 And no training job row should be persisted

Scenario: Model cannot exist without successful training job
 When the system attempts to register a model without a valid training job id
 Then the database should reject the operation
 And no model row should be persisted

Scenario: Dataset names are unique
 Given a dataset named "desk-objects-v1" exists
 When the system attempts to create another dataset named "desk-objects-v1"
 Then the database should reject the duplicate name

Scenario: Dataset version names are unique per dataset
 Given dataset "desk-objects-v1" has version "v1"
 When the system attempts to create another version "v1" for the same dataset
 Then the database should reject the duplicate version

Scenario: Same version name can exist across different datasets
 Given dataset "desk-objects-v1" has version "v1"
 And dataset "warehouse-objects-v1" exists
 When the system creates version "v1" for dataset "warehouse-objects-v1"
 Then the database should accept the operation

21. Feature - Observability

@p1 @api @worker @nonfunctional
 Feature: Observability
 The platform must expose enough information to debug requests, jobs and failures.

Background:
 Given the PerceptionLab API is running

Scenario: API response contains request id
 When I call GET "/health"
 Then the response should contain a request id header

Scenario: Failed API request returns structured error
 When I create a dataset with an invalid payload
 Then the response status should be 422
 And the response should contain an error code
 And the response should contain a human-readable message
 And the response should not contain a stack trace

Scenario: Training job failure is traceable
 Given a training job fails
 When I retrieve the training job details
 Then the response should contain the job status "failed"
 And the response should contain an error message
 And the response should contain timestamps for created_at, started_at and finished_at

Scenario: Worker logs job lifecycle events
 Given a training job exists
 When the worker processes the training job
 Then the logs should contain the job id

And the logs should contain status transition from "queued" to "running"
And the logs should contain final status

22. Feature - Performance Smoke

```
@p1 @nonfunctional @performance
Feature: Performance smoke tests
  The platform must meet minimal performance expectations for the MVP demo.

  Background:
    Given the PerceptionLab stack is running locally
    And a dataset named "desk-objects-v1" exists

  Scenario: API creates dataset quickly
    When I create a valid dataset
    Then the response time should be below 500 milliseconds

  Scenario: API accepts concurrent image uploads
    When I upload 20 valid images concurrently
    Then all uploads should complete successfully
    And no upload response should be 500
    And every uploaded image should have metadata in the database

  Scenario: Training job creation is asynchronous
    Given a trainable dataset version exists
    When I create a training job
    Then the response time should be below 500 milliseconds
    And the job status should be "queued"
    And the HTTP request should not wait for training completion

  Scenario: Inference returns within acceptable local MVP latency
    Given a registered tiny model exists
    When I run inference on image "cup.jpg"
    Then the response time should be below 1000 milliseconds
    And the response should contain "latency_ms"
```

23. Scénario end-to-end principal

Ce scénario est le plus important du projet. Il doit être visible dans le README et passer en local.

```
@p0 @e2e @api @storage @database @worker @ml
Feature: End-to-end computer vision pipeline
  PerceptionLab must transform raw visual data into a trained and usable model.

  Scenario: Build a vision model from ingested data
    Given the PerceptionLab stack is running
    And the database is clean
    When I create a dataset named "desk-objects-v1" with task type "object_detection"
    And I define classes:
      | cup |
      | book |
      | phone |
    And I upload images:
      | cup.jpg |
      | book.jpg |
      | phone.jpg |
    And I add valid annotations for each uploaded image
    And I create dataset version "v1"
    And I create a training job from dataset version "v1"
    Then the training job status should be "queued"

    When the worker processes the training job in fake training mode
    Then the training job status should become "succeeded"
    And training metrics should be available
```

```
And a model should be registered
And the model should have status "candidate"

When I run inference with the model on image "cup.jpg"
Then the inference response should contain detections
And each detection should contain class name, confidence and bounding box

When I generate a visual overlay from the inference result
Then an overlay image should be created
And the overlay image should exist in storage
```

24. Test BDD anti-régression critique

Ces scénarios doivent toujours passer. Ce sont les garde-fous du produit.

```
Créer dataset valide
Uploader image valide
Refuser fichier invalide
Ajouter annotation valide
Refuser bbox invalide
Créer dataset version
Garantir immutabilité dataset version
Créer training job queued
Worker passe job running
Worker passe job succeeded
Metrics disponibles
Model registry créé
Inference retourne detections
Storage et DB restent cohérents
```

25. Contrats d'erreur standardisés

Tous les endpoints doivent retourner des erreurs structurées.

Format recommandé:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request payload",
    "details": [
      {
        "field": "name",
        "issue": "Dataset name cannot be empty"
      }
    ],
    "request_id": "req_01hxyz"
  }
}
```

Codes d'erreur attendus:

```
VALIDATION_ERROR
NOT_FOUND
CONFLICT
UNSUPPORTED_MEDIA_TYPE
PAYLOAD_TOO_LARGE
STORAGE_ERROR
DATABASE_ERROR
TRAINING_ERROR
INFERENCE_ERROR
```

```
EXPORT_ERROR
INTERNAL_ERROR
```

BDD générique:

```
@p0 @api
Feature: Standard API errors
  API errors must be predictable and safe.

  Scenario: Validation error response follows standard format
    When I send an invalid request payload
    Then the response status should be 422
    And the response should contain "error.code"
    And the response should contain "error.message"
    And the response should contain "error.request_id"

  Scenario: Internal errors do not expose stack traces
    Given the API encounters an internal error
    When I call the failing endpoint
    Then the response status should be 500
    And the response should contain a safe error message
    And the response should not contain internal stack traces
    And the response should not contain file system paths
```

26. Contrôles spécifiques ML

Les tests ML ne doivent pas chercher à prouver que le modèle est excellent. Ils doivent prouver que le pipeline ML est correct.

QA ML MVP

```
Le worker peut charger une dataset version.
Le worker peut matérialiser les images et annotations.
Le worker peut lancer un training fake ou tiny.
Le worker écrit les métriques.
Le worker produit un artefact modèle.
Le registry référence l'artefact.
L'inférence retourne un contrat JSON stable.
```

QA ML V2

```
Le modèle dépasse un seuil mAP50 minimal sur dataset seed.
L'export ONNX est exécutable.
L'inférence ONNX retourne un format équivalent.
Les classes du modèle correspondent aux classes du dataset snapshot.
```

BDD ML:

```
@p1 @ml
Feature: ML pipeline consistency
  The ML worker must preserve dataset and model consistency.

  Scenario: Model classes match dataset version classes
    Given a dataset version contains classes:
      | cup |
      | book |
    When a model is trained from this dataset version
    Then the registered model should contain classes:
      | cup |
      | book |
```

```
Scenario: Training metrics are linked to model artifact
  Given a successful training job produced metrics
  When the model is registered
  Then the model metrics summary should match the training job final metrics
```

```
Scenario: Inference classes must come from model metadata
  Given a model was trained with classes:
    | cup |
    | book |
  When I run inference with this model
  Then every detection class should be one of:
    | cup |
    | book |
```

27. QA base de données

Même avec des tests API, il faut une QA DB directe, parce que le projet repose sur la traçabilité.

Contrôles DB à prévoir:

```
Migrations up/down
Contraintes foreign keys
Contraintes unique
Contraintes NOT NULL
Contraintes enum/status
Index sur dataset_id, sample_id, job_id, model_id
Absence de metadata orpheline
Absence d'artefacts non référencés après succès
Cohérence entre job status et model creation
```

Tests DB minimaux:

```
Un sample sans dataset est impossible.
Une annotation sans sample est impossible.
Un job sans dataset_version est impossible.
Un modèle sans training_job réussi est impossible.
Deux datasets ne peuvent pas avoir le même nom.
Deux versions d'un même dataset ne peuvent pas avoir le même nom.
Une dataset_version ne doit pas être modifiée après création.
```

28. QA storage

Le storage doit être testé comme un composant produit, pas juste comme un détail technique.

Contrôles:

```
Upload sample écrit bien un objet.
Checksum calculé.
Metadata créée après succès storage.
Suppression ou rollback si erreur storage.
Model artifact stocké après training.
Export artifact stocké après export.
Overlay artifact stocké après génération.
Download artifact possible.
```

Règle importante:

No database row should claim that an artifact exists if the artifact was not successfully written.

29. QA worker

Le worker PyTorch est un composant critique.

Contrôles:

```
Il récupère uniquement les jobs queued.
Il lock un job avant exécution.
Il ne traite pas deux fois le même job.
Il passe queued -> running -> succeeded.
Il passe queued -> running -> failed en cas d'erreur.
Il écrit started_at et finished_at.
Il écrit les métriques.
Il crée le modèle uniquement si succès.
Il conserve une erreur lisible si échec.
```

BDD complémentaire:

```
@p0 @worker
Feature: Worker job locking
  Training jobs must not be processed twice.

  Scenario: Two workers cannot process the same job
    Given a training job exists with status "queued"
    And two workers poll for jobs at the same time
    When the first worker locks the job
    Then the second worker should not receive the same job
    And only one worker should process the job
```

30. QA sécurité MVP

Pour le MVP local, pas besoin d'auth complexe, mais il faut éviter les erreurs grossières.

Scénarios sécurité:

```
@p1 @api @security
Feature: Basic API security
  The API must reject unsafe inputs.

  Scenario: Uploaded filename is sanitized
    Given a dataset exists
    When I upload a file named "../../../secret.jpg"
    Then the upload should not write outside the storage directory
    And the stored filename should be sanitized

  Scenario: Error response does not leak environment variables
    Given the API encounters an internal error
    When I call the failing endpoint
    Then the response should not contain environment variable values

  Scenario: API rejects unsupported content type
    When I send a JSON payload to an endpoint expecting multipart image upload
    Then the response status should be 415
```

31. Definition of Done QA

Une fonctionnalité est terminée seulement si:

- Les scénarios BDD P0 associés passent.
- Les tests unitaires critiques passent.
- Les tests d'intégration passent.
- Les erreurs sont structurées.
- Les logs utiles existent.
- Les règles DB sont respectées.
- La documentation API est à jour.
- Les fixtures de test sont disponibles.
- Le scénario est reproductible via Docker Compose.

Pour un module ML:

- Le mode `fake_training` fonctionne.
- Le mode `tiny_training` fonctionne au moins localement.
- Les métriques sont persistées.
- Les artefacts sont stockés.
- Les erreurs worker sont lisibles.
- Le registry est cohérent.

32. Plan d'exécution QA par sprint

Sprint 1 - API foundation QA

Objectif QA:

- Valider healthcheck, datasets, DB migrations, erreurs standardisées.

Scénarios à livrer:

- `health.feature`
- `dataset_management.feature`
- `database_integrity.feature` partiel
- `standard_api_errors.feature`

Gate de sprint:

- Création dataset robuste
- Validation payload
- DB propre
- Docker Compose démarre

Sprint 2 - Data ingestion QA

Objectif QA:

- Valider upload image, storage, metadata, annotations et dataset stats.

Scénarios à livrer:

```
sample_ingestion.feature
annotation_management.feature
artifacts_storage.feature partiel
```

Gate de sprint:

```
Upload valide OK
Fichiers invalides refusés
Annotations valides OK
Bbox invalides refusées
Stats dataset exactes
```

Sprint 3 - Versioning + Training QA

Objectif QA:

```
Valider dataset versioning, training jobs et worker lifecycle.
```

Scénarios à livrer:

```
dataset_versioning.feature
training_jobs.feature
metrics_tracking.feature
worker_job_locking.feature
```

Gate de sprint:

```
Dataset version immuable
Job async
Worker fake_training OK
Metrics écrites
Failed job propre
```

Sprint 4 - Registry + Inference QA

Objectif QA:

```
Valider model registry, inference et overlay.
```

Scénarios à livrer:

```
model_registry.feature
inference_api.feature
visual_overlay.feature
```

Gate de sprint:

```
Training succeeded crée modèle
Inference retourne JSON stable
Overlay généré
Artefacts cohérents
```

Sprint 5 - Export + Polish QA

Objectif QA:

```
Valider export ONNX, performance smoke et démo README.
```

Scénarios à livrer:

```
model_export.feature  
performance_smoke.feature  
end_to_end_pipeline.feature
```

Gate de sprint:

```
E2E complet vert  
ONNX export disponible ou proprement mocké  
Benchmark minimal  
README testable
```

33. Résultat attendu pour le repo

À la fin de cette passe QA BDD, le repo doit contenir:

```
docs/  
  qa-bdd.md  
  test-strategy.md  
  acceptance-matrix.md  
  
qa/  
  features/  
    *.feature  
  
fixtures/  
  images/  
  annotations/  
  payloads/  
  
scripts/  
  run_bdd.sh  
  seed_demo_dataset.sh
```

Commandes attendues:

```
docker compose up -d  
./scripts/run_bdd.sh
```

Output attendu:

```
P0 scenarios: passed  
P1 scenarios: passed or documented  
E2E pipeline: passed
```

34. Synthèse

Le rôle de cette QA BDD est de transformer PerceptionLab en projet crédible et défendable.

Elle doit prouver trois choses:

Le produit respecte les comportements attendus.
L'infrastructure est cohérente et robuste.
Le pipeline ML est traçable de la donnée brute au modèle exploitable.

Le scénario qui résume tout:

I can ingest visual data, annotate it, version it, train a model, register it, run inference, and generate a visual result - all through a clean Rust + PyTorch platform.

La prochaine passe logique est la pass techno QA: choix du runner BDD, tests API, fixtures, Docker Compose test profile, fake worker, migrations de test, CI GitHub Actions et structure opérationnelle du dossier `qa/`.